Real-Time Logistics & Fleet Operations Intelligence

A local dispatcher system for a fictional South African logistics operator. It supports delivery and route planning, vehicle/driver assignment, simulated GPS telemetry, operational exceptions, recovery assignments and recorded journey replay.

100
streaming simulated vehicles

2
independent event consumers

2,200
durable processed events

Architecture and data flow

Simulation	Go journey owner emits GPS events; journey progress and sequence are checkpointed after acknowledged publication.
Redpanda	fleet.telemetry.v1 is partitioned by vehicle; consumer offsets are committed after processing effects.
history-v1	Validates and deduplicates events; persists telemetry to a TimescaleDB hypertable and records exceptions and outcomes.
live-v1	Updates a monotonic Redis projection and broadcasts new positions to WebSocket subscribers.
Dispatcher	React, TypeScript and MapLibre display operational state; a periodic snapshot reconciles missed messages.

Operational workflow

Plan and dispatch. Create deliveries, sequence stops, allocate available vehicles and drivers, and start journeys. Relational constraints prevent conflicting active assignments.

Observe and intervene. Deviation, deadline, prolonged-stop and breakdown detection creates persisted alerts. Recovery reassignment records the reason and preserves the earlier assignment history.

Inspect history. Journey replay reads chronological TimescaleDB samples in pages and does not modify the live operation.

Consistency, testing & limits

Consistency controls

At-least-once streaming. A durable processed-event ledger guards historical side effects. Kafka/Redpanda delivery is not described as end-to-end exactly once.

Ordering. Vehicles are partition keys. Stale GPS can be stored for history, but the live position and operational state advance only for newer timestamps and sequences.

Failures. Invalid schema or journey-context events are written to a dead-letter topic. Dependency failures are retried with bounded backoff. Slow WebSocket clients can recover using an API snapshot.

Durable operations. Recovery assignment changes are transactional; backend restart pauses active journeys for explicit dispatcher review.

Executed verification · 3 October 2026

Streaming	100 simulated vehicles delivered real positions through Redpanda and the WebSocket service.
Operations	Route planning, dispatch, conflicting assignments, recovery reassignment, all four exception detectors and two delivery completions passed.
Data	A real TimescaleDB hypertable, duplicate/stale-GPS handling, 2,200 durable events and persisted journey replay were verified.
Recovery	Backend, Redis and PostgreSQL restarts preserved durable records; both consumer groups caught up.
Observability	Prometheus scrape, broker lag metrics, Grafana dashboard and frontend live stream/replay checks passed.

Deployment boundaries

The validated deployment is a local Docker Compose stack. Kubernetes resource manifests were structurally checked, but no cluster was deployed. A single Go process owns the simulator and WebSocket hub; Redpanda and PostgreSQL each run without high availability. Fixture routing is deterministic geometry, not road-network routing or optimisation.

The demonstration has no identity provider or tenant isolation. Public deployment would require authorization, TLS, durable backup/retention policy, multi-replica ownership changes and production-scale tests. The OpenStreetMap basemap also requires an internet connection.